



UNIVERSIDAD DE SAN CARLOS DE GUATEMALA FACULTAD DE
INGENIERÍA LABORATORIO INTRODUCCION A LA
PROGRAMACION Y COMPUTACION 2
VACACIONES 1MER SEMESTRE 2026
Auxiliar: FERNANDO JOSE VICENTE VELASQUEZ

• Actividad de Investigación y
Práctica: Estructuras de Datos
Avanzadas y APIs con ASP.NET Core

Nombre: Eldrick Aldair Hernández Bautista_____

Carnet:

2	0	2	4	–	0	3	4	8	8
---	---	---	---	---	---	---	---	---	---

Nota	
------	--

Parte 1: Investigación Teórica 1.

Estructuras de Datos Eficientes Árboles Binarios de Búsqueda (ABB): Su regla de ordenamiento establece que para cualquier nodo, los valores en su subárbol izquierdo son menores que el nodo raíz, y los del subárbol derecho son mayores. Su principal desventaja es que, si los datos se insertan en orden secuencial, el árbol no se balancea y degenera en una lista vinculada, perdiendo su eficiencia y pasando de un tiempo de búsqueda óptimo a una complejidad lineal de $O(n)$.

Árboles AVL: Es un árbol binario de búsqueda auto-balanceado, lo que significa que ajusta su estructura automáticamente al insertar o eliminar nodos para mantener una altura mínima. Su factor de balanceo se calcula como (Altura Izquierda - Altura Derecha). Gracias a este factor, la diferencia de altura entre subárboles nunca excede 1, garantizando que el árbol se mantenga poco profundo y su complejidad para búsqueda, inserción y eliminación sea siempre $O(\log n)$.

2. Fundamentos de Web APIs

API y Modelo Cliente-Servidor: Una API es una interfaz que permite la comunicación entre sistemas. En el modelo Cliente- Servidor, el cliente envía una petición (Request) al servidor mediante el protocolo HTTP. El servidor procesa la solicitud y devuelve una respuesta (Response). Verbos HTTP: - GET: Se utiliza para recuperar recursos. Es un método idempotente, lo que significa que realizar la misma petición varias veces producirá el mismo resultado sin modificar el estado del servidor. - POST: Se usa para crear nuevos recursos. No es idempotente, ya que hacer la misma petición múltiples veces creará múltiples recursos nuevos.

Parte 2:

Implementación Práctica en C# con ASP.NET Core A continuación, se presenta el código de la API utilizando un arreglo estático para el almacenamiento en memoria, tal como se implementó en el archivo Program.cs:

```
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Http;
using System.Linq;
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

// 3. Configuración del almacenamiento con ARREGLO ESTÁTICO

var coleccionNodos = new NodoElemento[100];
int cantidadActual = 0;
```

```

coleccionNodos[cantidadActual] = new NodoElemento { Id = 10, Valor =
    "Raíz Inicial (ABB)" };
cantidadActual++;
coleccionNodos[cantidadActual] = new NodoElemento { Id = 5, Valor =
    "Hijo Izquierdo" };
cantidadActual++;

// 4. Implementación de Endpoints
app.MapGet("/api/nodos", () => {
    var nodosOcupados = coleccionNodos.Take(cantidadActual);
    return Results.Ok(nodosOcupados);
});

app.MapPost("/api/nodos", (NodoElemento nuevoNodo) => {
    {
        if (cantidadActual >= coleccionNodos.Length)
        {
            return Results.BadRequest("El arreglo está lleno. No se pueden
            agregar más nodos.");
        }
        if (nuevoNodo.Id <= 0 || string.IsNullOrEmpty(nuevoNodo.Valor))
        {
            return Results.BadRequest("Datos del nodo inválidos.");
        }
        coleccionNodos[cantidadActual] = nuevoNodo;
        cantidadActual++;
        return Results.Created($"/api/nodos/{nuevoNodo.Id}", nuevoNodo);
    }
});

app.Run();

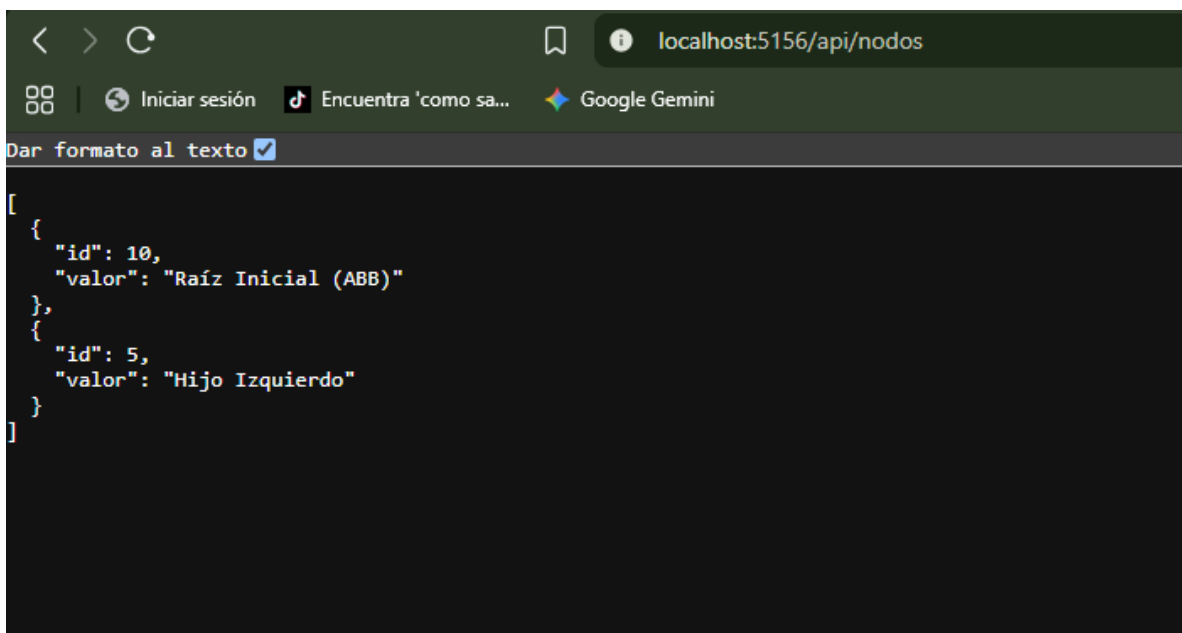
// 2. Modelado del Recurso
public class NodoElemento

```

```
{  
  
public int Id { get; set; }  
  
public string Valor { get; set; } = string.Empty;  
  
}
```

Parte 3:

Verificación y Pruebas A continuación, se presentan las evidencias de las pruebas realizadas a los endpoints de la API:



```
[  
  {  
    "id": 10,  
    "valor": "Raíz Inicial (ABB)"  
  },  
  {  
    "id": 5,  
    "valor": "Hijo Izquierdo"  
  }  
]
```

Program.cs

Extension: REST Client

pruebas.http X

pruebas.http > POST /api/nodos

1 ### Prueba del GET (Ver los nodos iniciales)

Send Request

2 GET http://localhost:5156/api/nodos

3

4 ###

5

6 ### Prueba del POST (Insertar un nuevo nodo)

Send Request

7 POST http://localhost:5156/api/nodos

8 Content-Type: application/json

9

10 {

11 | "id": 15,

12 | "valor": "Nuevo Nodo Derecho"

13 | }

```

4
5  var builder = WebApplication.CreateBuilder(args);
6  var app = builder.Build();
7
8
9  var coleccionNodos = new NodoElemento[100];
10 int cantidadActual = 0; // Llevamos el control de cuántos nodos hay realmente
11
12 // Insertamos los valores iniciales usando el índice
13 coleccionNodos[cantidadActual] = new NodoElemento { Id = 10, Valor = "Raíz Inicial (ABB)" };
14 cantidadActual++; // Incrementamos el contador
15
16 coleccionNodos[cantidadActual] = new NodoElemento { Id = 5, Valor = "Hijo Izquierdo" };
17 cantidadActual++;
18
19
20 app.MapGet("/api/nodos", () => {
21     var nodosOcupados = coleccionNodos.Take(cantidadActual);
22     return Results.Ok(nodosOcupados);
23 });
24
25
26 // POST: Inserta un nuevo nodo
27 app.MapPost("/api/nodos", (NodoElemento nuevoNodo) =>
28 {
29     // Validación 1: ¿Queda espacio en el arreglo?
30     if (cantidadActual >= coleccionNodos.Length)
31     {
32         return Results.BadRequest("El arreglo está lleno. No se pueden agregar más nodos.");
33     }
34
35     // Validación 2: Datos correctos
36     if (nuevoNodo.Id <= 0 || string.IsNullOrEmpty(nuevoNodo.Valor))
37     {
38         return Results.BadRequest("Datos del nodo inválidos.");
39     }
40
41     // Insertamos en la posición libre actual y luego aumentamos el contador
42     coleccionNodos[cantidadActual] = nuevoNodo;
43     cantidadActual++;
44
45     return Results.Created($"/api/nodos/{nuevoNodo.Id}", nuevoNodo);
46 });
47

```

```

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var coleccionNodos = new NodoElemento[100];
int cantidadActual = 0; // Llevamos el control de cuántos nodos hay realmente

// Insertamos los valores iniciales usando el índice
coleccionNodos[cantidadActual] = new NodoElemento { Id = 10, Valor = "Raíz Inicial (ABB)" };
cantidadActual++; // Incrementamos el contador

coleccionNodos[cantidadActual] = new NodoElemento { Id = 5, Valor = "Hijo Izquierdo" };
cantidadActual++;

app.MapGet("/api/nodos", () => {
    var nodosOcupados = coleccionNodos.Take(cantidadActual);
    return Results.Ok(nodosOcupados);
});

// POST: Inserta un nuevo nodo
app.MapPost("/api/nodos", (NodoElemento nuevoNodo) =>
{
    // Validación 1: ¿Queda espacio en el arreglo?
    if (cantidadActual >= coleccionNodos.Length)
    {
        return Results.BadRequest("El arreglo está lleno. No se pueden agregar más nodos.");
    }

    // Validación 2: Datos correctos
    if (nuevoNodo.Id <= 0 || string.IsNullOrEmpty(nuevoNodo.Valor))
    {
        return Results.BadRequest("Datos del nodo inválidos.");
    }

    // Insertamos en la posición libre actual y luego aumentamos el contador
    coleccionNodos[cantidadActual] = nuevoNodo;
    cantidadActual++;
}

```